

Documento 01: Arquitectura Front-End y Componentes del Sistema ASCEND

1. Introducción y Propósito del Documento

Este manual técnico explica de manera exhaustiva cómo está construido el Front-End (la interfaz visual y la interacción con el usuario) del **Panel de Administración** y del ecosistema general del **SGDM (Sistema de Gestión Deportiva Modular - ASCEND)**.

El objetivo es que cualquier desarrollador o evaluador comprenda exactamente la estructura de **HTML5 (HyperText Markup Language versión 5 - Lenguaje de Marcado de Hipertexto)**, **CSS3 (Cascading Style Sheets versión 3 - Hojas de Estilo en Cascada)** y **JavaScript Vanilla (código JavaScript nativo y puro sin librerías externas)** utilizado en la plataforma.

2. Glosario de Conceptos y Siglas Fundamentales

Para garantizar una comprensión total sin requerir búsquedas externas, a continuación se definen todos los términos técnicos utilizados:

- **UI (User Interface - Interfaz de Usuario):** Es el conjunto de elementos visuales (botones, campos de texto, formularios, barras de navegación, tablas) que permiten a una persona interactuar con el sistema informático.
- **UX (User Experience - Experiencia de Usuario):** Es la sensación de facilidad, agilidad, claridad y satisfacción que experimenta una persona al navegar y utilizar la plataforma.
- **DOM (Document Object Model - Modelo de Objetos del Documento):** Es la representación estructural en forma de árbol que el navegador web crea en memoria a partir del código HTML. Mediante JavaScript se puede manipular el DOM en tiempo real (crear elementos, cambiar textos, alternar clases CSS, eliminar filas de una tabla).
- **SPA (Single Page Application - Aplicación de Página Única):** Patrón de diseño web donde el usuario navega por diferentes secciones sin que el navegador recargue por completo la página web. En nuestro proyecto, implementamos una **simulación de SPA para maquetación** mediante el archivo `mockup-router.js`.
- **Mobile-First (Diseño Primero Móviles):** Filosofía de desarrollo en la cual los estilos visuales se programan primero para pantallas pequeñas de teléfonos celulares (evitando sobrecarga de ancho de banda y complejidad visual) y luego se van expandiendo progresivamente a pantallas de computadoras de escritorio mediante puntos de quiebre (*Breakpoints*).
- **CSS Custom Properties (Variables de CSS):** Mecanismo de CSS3 que permite declarar valores reutilizables (colores, tamaños de borde, sombras) en un único lugar global (`:root`) para que todo el sitio los consuma. Si se cambia el valor de una variable, todo el sistema visual se actualiza de inmediato.
- **Desacoplamiento (Decoupling):** Principio de ingeniería de software que consiste en mantener separadas las capas del sistema (el diseño visual por un lado, la lógica de interacción por otro, y el almacenamiento en base de datos por otro). Esto permite modificar el diseño sin romper el servidor ni la base de datos.

- **Event Bubbling (Propagación / Burbujeo de Eventos):** Mecanismo nativo de los navegadores donde un evento (como un clic en un botón) sube hacia los elementos contenedores padres en el DOM a menos que se detenga explícitamente con `event.stopPropagation()`.
- **Event Delegation (Delegación de Eventos):** Patrón de diseño en JavaScript que consiste en colocar un único escuchador de eventos (`EventListener`) en un elemento contenedor padre para gestionar las interacciones de todos sus elementos hijos (incluso aquellos que se crean dinámicamente en el futuro).

3. Estructura y Árbol de Archivos del Administrador

La capa visual y de interacción del administrador se encuentra organizada de forma modular:

SGDM/	
— codigo_fuente/	
— vistas/	
— admin/	<-- Vistas HTML (Plantillas y
pantallas)	
— index.html	<-- Contenedor maestro (Shell de la
aplicación)	
— dashboard.html	<-- Tablero principal con métricas y
resúmenes	
— usuarios/	<-- Módulo de usuarios administrativos
— lista.html	<-- Tabla con listado y filtros de
usuarios	
— formulario.html	<-- Formulario de creación y edición
— roles/	<-- Módulo de roles y permisos
— lista.html	
— formulario.html	
— juegos/	<-- Módulo de disciplinas deportivas /
eSports	
— lista.html	
— formulario.html	
— torneos/	<-- Módulo de torneos
— lista.html	
— crear.html	
— configurar.html	
— historial.html	
— equipos/	<-- Módulo de equipos y solicitudes
— lista.html	
— formulario.html	
— solicitudes.html	
— jugadores/	<-- Directorio de jugadores
registrados	
— lista.html	
— organizadores/	<-- Directorio de organizadores
— lista.html	
— resultados/	<-- Carga y validación de marcadores
— lista.html	
— formulario.html	
— registrar.html	
— comunicaciones/	<-- Envío de comunicados masivos
— enviar.html	

```

| | | └─ reportes/                                <-- Exportación de reportes
PDF/Excel/CSV
| | |   └─ exportar.html
| | |   └─ configuracion/                          <-- Políticas de seguridad y auditoría
| | |     │ general.html
| | |     │ auditoria.html
| | |     └─ accesos.html
| | |   └─ perfil/                                 <-- Edición de perfil de administrador
| | |     └─ editar-perfil.html
| └─ publico/
|   └─ css/
|     └─ admin/                                  <-- Arquitectura de Hojas de Estilo
CSS3
| | | └─ admin.css                               <-- Archivo central que unifica los
submódulos
| | | └─ base.css                               <-- Variables globales, tipografía y
resets
| | | └─ layout.css                             <-- Estructura visual: Header,
Sidebar, Main, Footer
| | | └─ componentes.css                       <-- Tablas, formularios, cards,
toasts, modales
| | | └─ utilidades.css                        <-- Botones, estados, badges, clases
auxiliares
| | | └─ perfil.css                            <-- Estilos específicos del perfil y
avatar
| | └─ js/
| |   └─ admin/                                <-- Lógica JavaScript Vanilla
| |     └─ admin.js                           <-- Menús móviles, acordeones,
validaciones
| | └─ mockup-router.js                       <-- Enrutador SPA para maquetación por
hash
| | └─ reportes.js                            <-- Gestión de descargas y eventos en
reportes
```

🧠 4. Arquitectura CSS Modular: Desglose Técnico

En lugar de utilizar un único archivo gigante de miles de líneas que sería incomprensible e inmantenible, se aplicó el principio de **Separación de Responsabilidades** unificado por [admin.css] (file:///c:/Users/juani/Documents/GitHub/SGDM/codigo_fuente/publico/css/admin/admin.css) mediante directivas `@import`:

4.1 [base.css]

(file:///c:/Users/juani/Documents/GitHub/SGDM/codigo_fuente/publico/css/admin/base.css)
(El Sistema de Diseño y Tokens)

Declara las variables CSS en la pseudo-clase `:root` (el elemento raíz del documento HTML). Esto define la paleta corporativa oficial de ASCEND:

```

:root {
  /* Paleta de Colores Corporativa */
  --color-fondo-principal: #f8f9fa; /* Gris ultra claro para descanso visual
del fondo */
  --color-fondo-panel: #ffffff; /* Blanco puro para tarjetas y tablas */
  --color-fondo-sidebar: #212529; /* Gris oscuro profesional para la barra
de navegación */
  --color-texto-principal: #333333; /* Gris carbón de alto contraste para
lectura */
  --color-texto-secundario: #6c757d; /* Gris medio para textos de ayuda y
etiquetas */
  --color-primario: #4b6584; /* Azul acero corporativo para elementos
principales */
  --color-primario-hover: #3c526a; /* Azul oscuro para el estado de pasar el
cursor */
  --color-exito: #45b676; /* Verde esmeralda para estados positivos
y triunfos */
  --color-peligro: #e05d5d; /* Rojo carmesí para botones de
eliminación o alertas */
  --color-advertencia: #f39c12; /* Ámbar para estados pendientes */

  /* Tipografía y Espaciados */
  --fuente-principal: 'Montserrat', sans-serif;
  --borde-radio: 6px; /* Bordes ligeramente redondeados y
modernos */
  --sombra-suave: 0 4px 6px rgba(0, 0, 0, 0.05); /* Sombra elegante que da
profundidad */
}

```

- **Reset Universal:** Aplica `box-sizing: border-box;` a todos los elementos `*` para que el ancho y alto incluyan el *padding* (relleno interno) y el borde, evitando desbordamientos no deseados en la pantalla.
- **Suavizado de Fuentes:** Se utiliza `-webkit-font-smoothing: antialiased;` para que los textos en pantallas de alta densidad se lean nítidos y sin distorsión.

4.2 [layout.css]

(file:///c:/Users/juani/Documents/GitHub/SGDM/codigo_fuente/publico/css/admin/layout.css)
(La Estructura Espacial del Panel)

Controla la disposición espacial de la aplicación utilizando **CSS Flexbox (Modelo de Caja Flexible)** y **Media Queries**:

- **Mobile-First Layout:** Por defecto (pantallas de celular de menos de 768 píxeles de ancho), la barra lateral `.menu-lateral` está oculta fuera de la pantalla mediante `transform: translateX(-100%);` con una transición fluida `transition: transform 0.3s ease;`. Al presionar el botón hamburguesa, JavaScript le añade la clase `.activo`, aplicando `transform: translateX(0);` y haciéndola deslizar suavemente hacia la vista.
- **Escritorio (@media (min-width: 768px)):** Al detectar una pantalla ancha, la barra lateral pasa a tener una posición fija (`position: fixed; width: 250px; height: 100vh;`) y el área central `.area-`

`contenido` adquiere un margen izquierdo `margin-left: 250px;`, ocupando todo el ancho restante sin solapamientos.

- **Header Superior (.cabecera-principal):** Barra superior fija que contiene el logotipo del sistema, el botón de colapso de menú y el menú desplegable del perfil del administrador.
-

4.3 [componentes.css]

(file:///c:/Users/juani/Documents/GitHub/SGDM/codigo_fuente/publico/css/admin/componentes.css) (Piezas de Interfaz Reutilizables)

Define los componentes visuales que se repiten a lo largo de todo el panel:

1. **Tablas Responsivas (.tabla-estandar):** Envueltas en contenedores con `overflow-x: auto;` para que en teléfonos móviles el usuario pueda desplazarse horizontalmente sin que la página se rompa ni genere barras de desplazamiento indeseadas en toda la ventana. Cada fila posee un efecto de iluminado sutil al pasar el cursor (`:hover`).
 2. **Formularios Semánticos (.formulario-estandar):** Utilizan etiquetas `<fieldset>` (para agrupar lógicamente secciones de datos) y `<legend>` (para titular cada bloque). Los campos de texto (`<input>`, `<select>`, `<textarea>`) tienen efectos de foco (*focus ring*) con sombras azuladas (`box-shadow: 0 0 0 3px rgba(75, 101, 132, 0.2);`) para guiar visualmente al usuario mientras escribe.
 3. **Tarjetas Métricas KPI (Key Performance Indicators) (.tarjeta-estadistica):** Muestran las estadísticas del tablero (Torneos Activos, Usuarios Registrados, Equipos). Poseen una microinteracción de elevación (`transform: translateY(-2px);`) cuando el cursor pasa por encima.
 4. **Menús de Acciones en Tablas (.acciones-desplegables):** Implementados de forma moderna y nativa con los elementos HTML5 `<details>` y `<summary>`. No requieren plugins externos; al hacer clic, se despliega una lista contextual con opciones (Ver como, Editar, Suspende, Eliminar) animada con `@keyframes desplegarSuave`.
 5. **Sistema de Alertas Toast Flotantes (.notificacion-toast):** Mensajes emergentes temporales que aparecen en la esquina superior derecha para confirmar acciones (ej. "Cambios guardados con éxito"). Están diseñados con `@keyframes deslizarArriba` y `@keyframes desvanecer` para una salida elegante.
-

4.4 [utilidades.css]

(file:///c:/Users/juani/Documents/GitHub/SGDM/codigo_fuente/publico/css/admin/utilidades.css) (Clases Atómicas y Botones)

Proporciona clases de ayuda reutilizables:

- **Botones Semánticos:**
 - `.boton-accion` / `.boton-primario`: Azul corporativo para la acción principal.
 - `.boton-secundario`: Gris neutro para cancelar o retroceder.
 - `.boton-exito`: Verde esmeralda para guardar, aprobar o publicar.
 - `.boton-peligro`: Rojo carmesí para eliminar, rechazar o suspender.
 - **Badges / Etiquetas de Estado:**
 - `.estado-exito` / `.etiqueta-estado.activa`: Fondo verde translúcido con texto verde fuerte.
 - `.estado-fallido` / `.etiqueta-estado.inactiva`: Fondo rojo translúcido.
 - `.estado-pendiente`: Fondo ámbar translúcido.
-

⚡ 5. Lógica JavaScript Vanilla: Desglose de Archivos y Métodos

Todo el comportamiento del panel está construido en JavaScript nativo (sin dependencias como jQuery o React), lo que asegura una ejecución instantánea en cualquier navegador.

5.1 [admin.js]

(file:///c:/Users/juani/Documents/GitHub/SGDM/codigo_fuente/publico/js/admin/admin.js) (El Cerebro de la Interfaz)

Este script se ejecuta cuando el DOM está completamente parseado gracias al evento `DOMContentLoaded`:

```
document.addEventListener('DOMContentLoaded', () => {  
  // Inicialización de componentes del panel  
});
```

Funcionalidades implementadas en `admin.js`:

1. Control del Menú Móvil (Hamburguesa):

- **Elemento:** `#btn-menu-movil` y `.menu-lateral`.
- **Mecanismo:** Al hacer clic en el botón, se añade o quita la clase `.activo` a la barra lateral.
- **Cierre Inteligente por Clic Exterior:** Se añade un `click` global a `document`. Si el usuario hace clic fuera de la barra lateral mientras está abierta, el script detecta que el clic no ocurrió dentro de `.menu-lateral` ni en el botón hamburguesa, y automáticamente remueve la clase `.activo`.
- **Método clave:** `evento.stopPropagation()` para evitar que el clic dentro del menú lo cierre a sí mismo.

2. Menú Desplegable de Perfil del Administrador:

- **Elemento:** `#btn-perfil-admin` y `#menu-perfil-admin`.
- **Mecanismo:** Alterna la clase CSS `.mostrar` para exhibir las opciones rápidas (Mi Perfil, Configuración, Cerrar Sesión). Al hacer clic en cualquier otra parte de la pantalla, se cierra automáticamente.

3. Submenú Tipo Acordeón en la Barra Lateral:

- **Elementos:** Enlaces de menú con clase `.menu-enlace` que tienen un submenú hermano `.menu-sublista`.
- **Comportamiento:** Al hacer clic en una categoría (por ejemplo "Torneos" o "Configuración"), el script alterna la visibilidad de su sublista (`display: block / none`).
- **Cierre de Vecinos (Acordeón Exclusivo):** Antes de abrir el submenú seleccionado, el script busca todos los demás submenús abiertos y los cierra, manteniendo el menú limpio y ordenado.

4. Validación de Formularios en el Cliente:

- **Elementos:** Formularios con la clase `.validacion-activa` y campos obligatorios con la clase `.campo-requerido`.

- **Comportamiento:** Al dispararse el evento `submit`, el script recorre todos los campos requeridos con un bucle `forEach`. Si un campo está vacío (`campo.value.trim() === ''`), cancela el envío con `evento.preventDefault()`, le agrega la clase `.campo-error` (que dibuja un borde rojo de advertencia) y muestra un mensaje de alerta guiando al usuario. Al momento en que el usuario empieza a escribir en ese campo (evento `input`), el borde rojo se remueve inmediatamente.

5. Protección de Acciones Destructivas (Confirmación de Borrado):

- **Elementos:** Botones con clase `.boton-peligro` o `.accion-eliminar`.
- **Mecanismo:** Intercepta el clic y lanza un cuadro de diálogo nativo `window.confirm("¿Está seguro de que desea realizar esta acción? Esta operación no se puede deshacer.")`. Si el usuario pulsa "Cancelar", la acción se aborta por completo.

5.2 [mockup-router.js]

(file:///c:/Users/juani/Documents/GitHub/SGDM/codigo_fuente/publico/js/admin/mockup-router.js) (El Enrutador de Maquetación)

En el desarrollo profesional, antes de programar en PHP sobre un servidor Apache/Nginx, es fundamental contar con una maqueta navegable que permita probar todas las pantallas. Este archivo implementa ese mecanismo.

¿Cómo funciona técnicamente?

1. **Detección del Hash de Navegación (`window.location.hash`):** Los enlaces de la barra lateral tienen el formato `<a href="#torneos/crear">`. El script escucha el evento `window.addEventListener('hashchange', cargarVista)` y también se ejecuta en la carga inicial (`window.addEventListener('DOMContentLoaded', ...)`).
2. **Petición Asíncrona con la API `fetch()`:** El router toma la ruta del hash (por ejemplo `torneos/crear`), la transforma en la ruta física local `torneos/crear.html` y realiza una petición HTTP asíncrona mediante `fetch()`.
3. **Inyección en el Contenedor Principal (`#area-contenido`):** Una vez recibido el texto HTML, el script actualiza el contenedor maestro: `document.getElementById('area-contenido').innerHTML = htmlObtenido;`
4. **Manejo de Estados de Pantalla (`en-inicio` vs `en-interna`):** Si la vista cargada es el `#dashboard`, agrega al `<body>` la clase `en-inicio` para mostrar el banner de bienvenida. Si se navega a cualquier submódulo interno, reemplaza la clase por `en-interna` para ocultar elementos de bienvenida y concentrarse en el formulario o tabla correspondiente.
5. **Simulación de Respuesta del Servidor y Toasts:** Intercepta los envíos de formularios con `formulario.addEventListener('submit', ...)`. Previene la recarga, simula un tiempo de espera de red de 800 milisegundos con `setTimeout()` y dispara una notificación flotante Toast (`crearToast("Operación realizada con éxito")`), permitiendo evaluar la experiencia de usuario de forma realista.

⚠ **Nota de Transición a PHP:** Este archivo `mockup-router.js` cumplió su ciclo en la etapa de maquetación. En la siguiente fase será sustituido por el motor de enrutamiento del servidor en PHP (*Front Controller*).

5.3 [reportes.js]

(file:///c:/Users/juani/Documents/GitHub/SGDM/codigo_fuente/publico/js/admin/reportes.js)
(Delegación de Eventos en Descargas)

Gestiona la interacción de los botones de exportación en la pantalla [reportes/exportar.html]

(file:///c:/Users/juani/Documents/GitHub/SGDM/codigo_fuente/vistas/admin/reportes/exportar.html).

- **Patrón Implementado: Delegación de Eventos en document:** Dado que la vista de reportes se carga dinámicamente mediante `fetch()`, si se colocaran los `addEventListener` directamente en los botones al inicio, estos no existirían en el DOM y fallarían. Para resolver esto, `reportes.js` escucha los clics en todo el documento y comprueba si el elemento pulsado coincide con los identificadores:
 - `#btn-exportar-pdf`
 - `#btn-exportar-excel`
 - `#btn-exportar-csv` Al detectarlo, valida si se seleccionó una opción en el `<select>` correspondiente y emite la notificación de descarga.
-

6. Resumen de Buenas Prácticas Aplicadas en el Front-End

1. **Semántica Estricta:** Cada etiqueta HTML se usa para su propósito natural (`<nav>` para navegación, `<aside>` para barra lateral, `<main>` para contenido central, `<form>` para recolección de datos, `<button type="submit">` para envíos).
2. **Cero Dependencias Bloqueantes:** No se descargan megabytes de librerías innecesarias; la carga es casi instantánea (menos de 50 milisegundos).
3. **Escalabilidad de Estilos:** Al usar variables CSS en `:root`, cualquier cambio de identidad visual o modo oscuro se realiza en minutos editando un solo archivo.
4. **Diseño Defensivo:** Formularios validados en el cliente, avisos de confirmación en borrados y tablas adaptadas para no desbordar en pantallas pequeñas.